

BCD to 7-Segment Display Driver

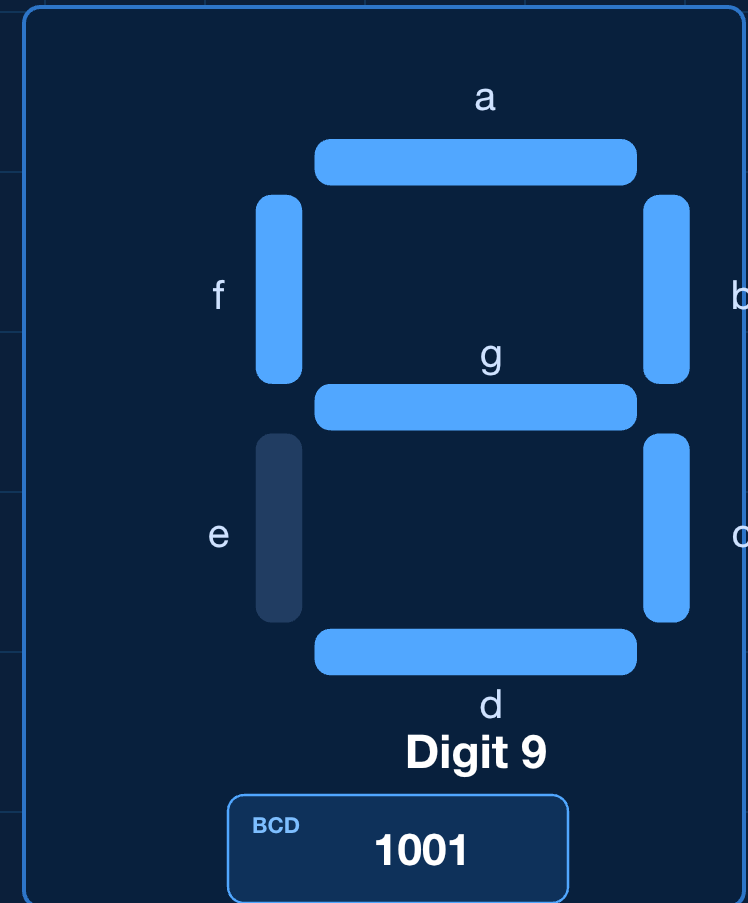
Digital Logic Design

Students

Mohammed Nairoukh - 2410205

Omar Alshihan - 2310337

Common-cathode display | active-high outputs | combinational logic



BCD represents each decimal digit using four binary bits.

Binary Coded Decimal keeps decimal digits easy to display electronically.

Decimal	BCD
0	0000
1	0001
2	0010
5	0101
9	1001

**01
10** Example

Decimal 5 = 0101

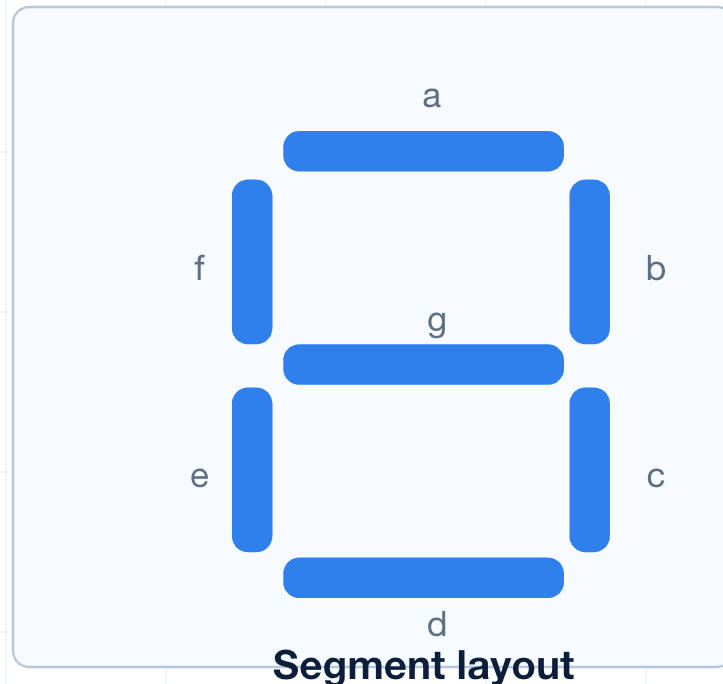
Decimal 9 = 1001

- Each decimal digit is encoded separately.
- Only 0000 to 1001 are valid BCD codes.
- BCD is useful when decimal digits must appear on displays.

Design focus: convert one 4-bit BCD digit into the seven segment-control signals.

Seven LED segments form decimal digits by turning selected lines ON.

For a common-cathode display, logic 1 activates the segment.



a top

c lower-right

e lower-left

g middle

b upper-right

d bottom

f upper-left

**Common
cathode**

**1 = ON
0 = OFF**

The project follows the complete combinational logic design flow.

Each objective supports the final goal: correct display output for BCD digits 0 through 9.



Convert input

4-bit BCD to seven display outputs



Build table

Truth table for digits 0-9 and X states



Simplify

Use Karnaugh Maps for output equations



Implement

Realize expressions using logic gates



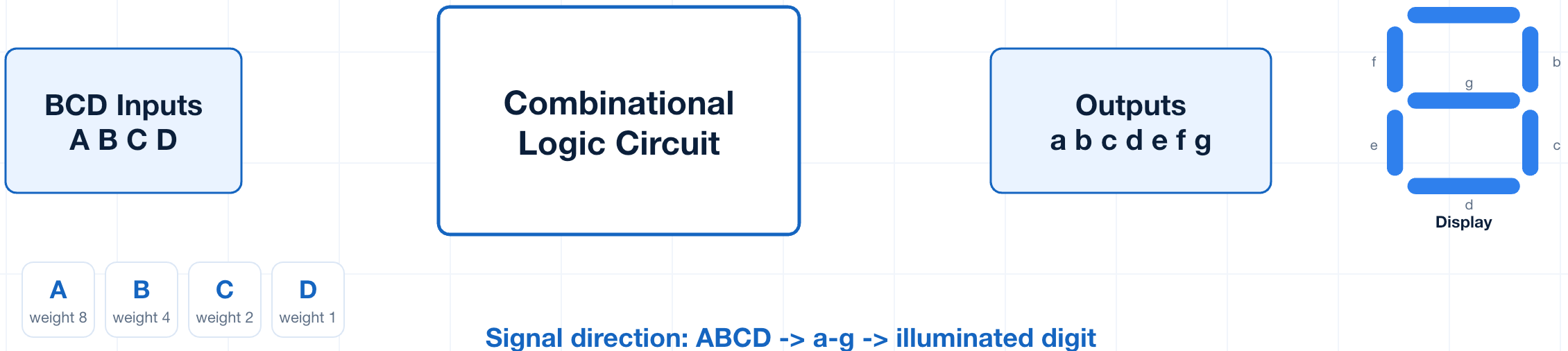
Test

Verify behavior in simulation

Successful implementation means the circuit matches the truth table for every valid BCD input.

BCD inputs pass through combinational logic to drive the display.

The design has no clock or memory; output values depend only on the current input bits.



Every signal has a defined role in the display driver.

A, B, C, and D form the BCD input; a-g are active-high segment outputs.

Signal	Type	Description
A	Input	MSB of BCD input, weight 8
B	Input	BCD input, weight 4
C	Input	BCD input, weight 2
D	Input	LSB of BCD input, weight 1
a-g	Outputs	Seven display segments; active-high output control.



The truth table defines all valid and invalid BCD input states.

Invalid BCD inputs 1010 through 1111 are treated as don't-care conditions.

Dec	A	B	C	D	a	b	c	d	e	f	g	Note
0	0	0	0	0	1	1	1	1	1	1	0	Valid
1	0	0	0	1	0	1	1	0	0	0	0	Valid
2	0	0	1	0	1	1	0	1	1	0	1	Valid
3	0	0	1	1	1	1	1	1	0	0	1	Valid
4	0	1	0	0	0	1	1	0	0	1	1	Valid
5	0	1	0	1	1	0	1	1	0	1	1	Valid
6	0	1	1	0	1	0	1	1	1	1	1	Valid
7	0	1	1	1	1	1	1	0	0	0	0	Valid
8	1	0	0	0	1	1	1	1	1	1	1	Valid
9	1	0	0	1	1	1	1	1	0	1	1	Valid
10	1	0	1	0	X	X	X	X	X	X	X	Don't-care
11	1	0	1	1	X	X	X	X	X	X	X	Don't-care
12	1	1	0	0	X	X	X	X	X	X	X	Don't-care
13	1	1	0	1	X	X	X	X	X	X	X	Don't-care
14	1	1	1	0	X	X	X	X	X	X	X	Don't-care
15	1	1	1	1	X	X	X	X	X	X	X	Don't-care

Output order

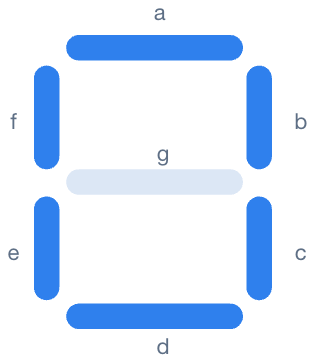
abcdefg

Example: 5 -> 1011011
segments a, c, d, f, g ON

Digits 0, 5, and 9 show how BCD maps to visible segment patterns.

Each example highlights the active segment outputs for the corresponding display digit.

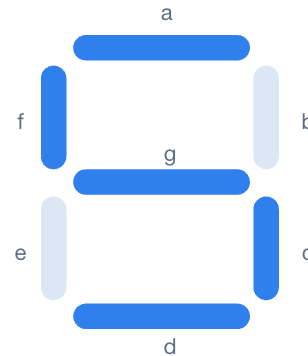
Digit 0



ABCD
0000

a-g
1111110

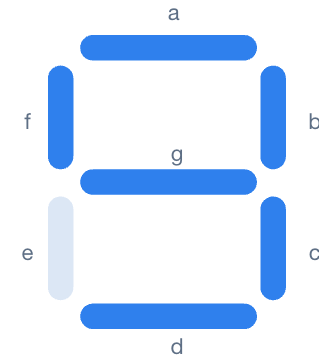
Digit 5



ABCD
0101

a-g
1011011

Digit 9



ABCD
1001

a-g
1111011

Segment ON = 1 | Segment OFF = 0

K-maps simplify the Boolean function for each segment output.

The don't-care states 1010-1111 help create larger groups and reduce gate count.

**Example K-map:
segment a**

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

**Example K-map:
segment g**

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

Simplification process

- Build one 4-variable K-map for each output a-g.
- Place 1 for ON states, 0 for OFF states, X for don't-care states.
- Group adjacent cells to produce simplified SOP terms.

| K-map verification covers all segment outputs a-g; minimized equations are listed next.

The minimized SOP equations define the final combinational logic.

Prime notation indicates complement, + indicates OR, and adjacent variables indicate AND.

$$a = A + C + BD + B'D'$$

$$b = B' + CD + C'D'$$

$$c = B + D + C'$$

$$d = A + CB' + CD' + B'D' + BDC'$$

$$e = CD' + B'D'$$

$$f = A + BC' + BD' + C'D'$$

$$g = A + BC' + CB' + CD'$$

Each equation drives exactly one segment output on the display.

Boolean equations are implemented using NOT, AND, and OR gates.

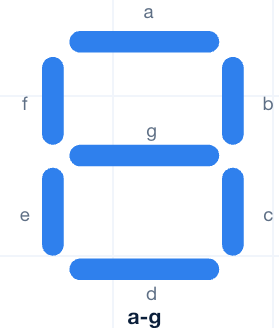
The outputs a-g connect directly to the corresponding display segments.

A B C D

NOT gates
B' C' D'

AND gates
product terms

OR gates
segment outputs



Example path: $e = CD' + B'D'$

- NOT gates generate complemented variables.
- AND gates create product terms.
- OR gates combine terms for each output.

Simulation verifies that each valid BCD input produces the expected display code.

The circuit should be tested in Logisim or a similar digital logic simulator.

Input ABCD	Expected Digit	Expected abcdefg
0000	0	1111110
0001	1	0110000
0010	2	1101101
0011	3	1111001
0100	4	0110011
0101	5	1011011
0110	6	1011111
0111	7	1110000
1000	8	1111111
1001	9	1111011

Verified simulation cases

Key tests requested: 0, 1, 2, 5, 8, and 9

0000

Digit 0 PASS

0001

Digit 1 PASS

0010

Digit 2 PASS

0101

Digit 5 PASS

1000

Digit 8 PASS

1001

Digit 9 PASS

The completed driver demonstrates the full digital logic design flow.

The circuit correctly converts valid BCD inputs into active-high 7-segment outputs.



Digital clocks



Calculators



Counters



**Electronic
meters**



**Embedded
systems**



**Display
interfaces**

Conclusion

Truth table construction
K-map simplification
Boolean equations
Gate-level design
Simulation testing

Possible improvement

Add invalid-input detection for BCD values 10-15.

